



第四章 程序的链接



第四章 程序的链接

一个大的程序往往会分成多个源程序来编写，因而需要对各个不同源程序文件分别进行编译或汇编。

把一个源程序称为一个模块。通过模块的划分，将复杂的问题合理分配到各子模块中去解决，便于多人合作，编程的质量和效率都会提高。

此外，通过模块的划分，可以针对不同模块的特性，采用不同的语言编程，有利于发挥各计算机语言的特性。





第四章 程序的链接

多个源程序会生成多个不同的目标代码文件。这些目标代码文件中包含指令、数据等信息。

此外，在程序中还会调用一些标准库函数，这些函数库也是一些目标代码文件。

因此，编译之后，需要将目标代码文件，包括用到的函数库目标文件，链接生成一个可执行文件。



主要内容



华中科技大学

- ◆ 4.1 编译、汇编和链接
- ◆ 4.2 目标文件格式
- ◆ 4.3 符号表和符号解析
- ◆ 4.4 重定位
- ◆ 4.5 可执行文件的加载

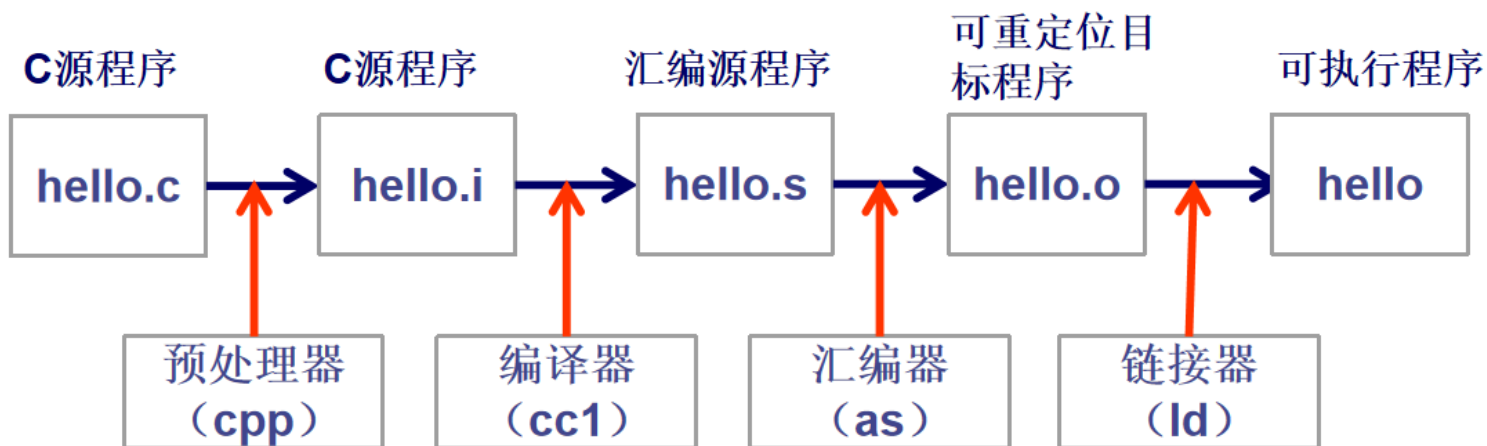




4.1.编译、汇编和链接

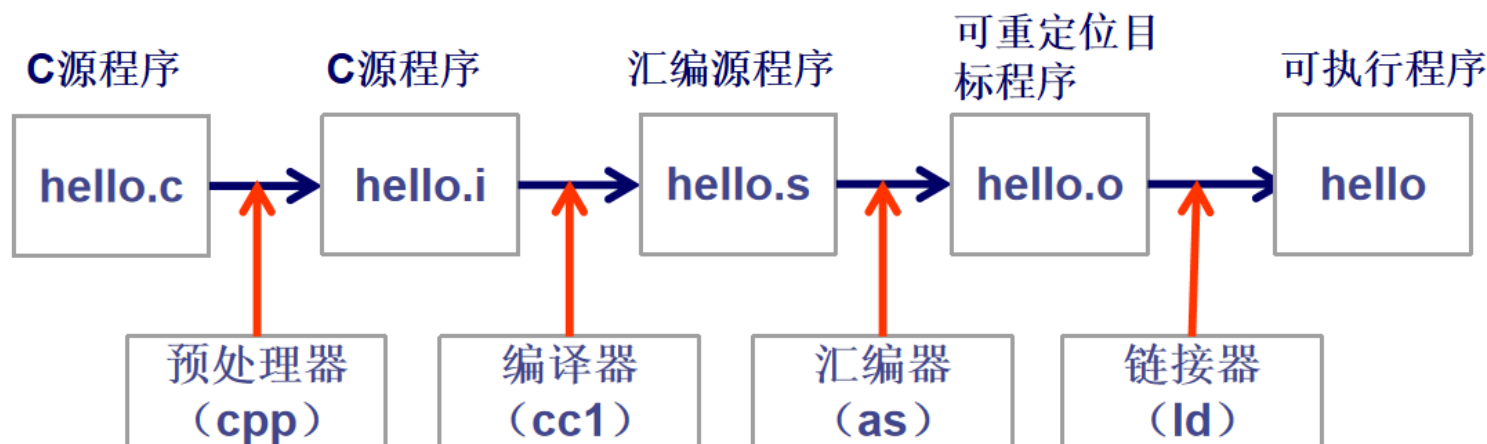
一、编译和汇编

将源程序转化为执行程序的过程：





一、编译和汇编



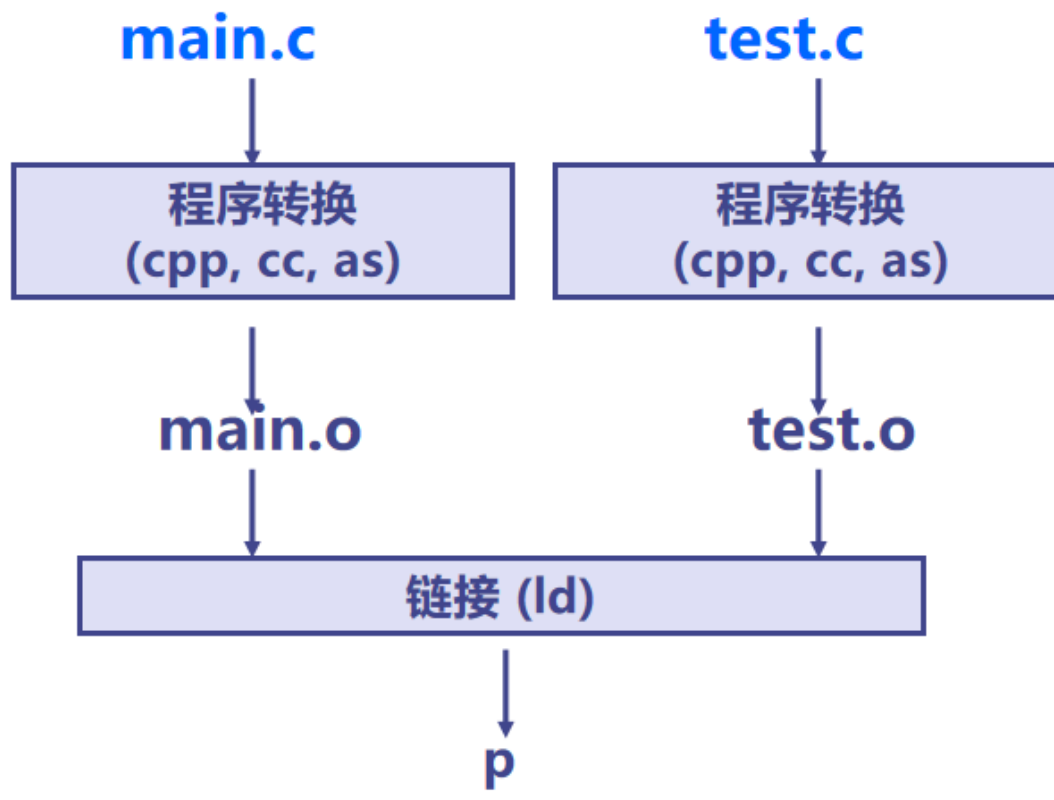
上述过程对应的命令为：

```
gcc hello.c -o hello.i -E  
gcc hello.i -o hello.s -S  
gcc hello.s -o hell.o -c  
gcc hello.o -o hello
```

二、链接

链接的功能：

将多个可重定位的目标文件合成一个可执行文件。





二、链接

例:

main.c:

```
int add(int ,int);
```

```
int main()
```

```
{
```

```
    return add(20, 13);
```

```
}
```

执行编译命令: gcc -c -m32 -
o main.o main.c

test.c:

```
int add(int i,int j) {
```

```
    int x=i+j;
```

```
    return x;
```

```
}
```

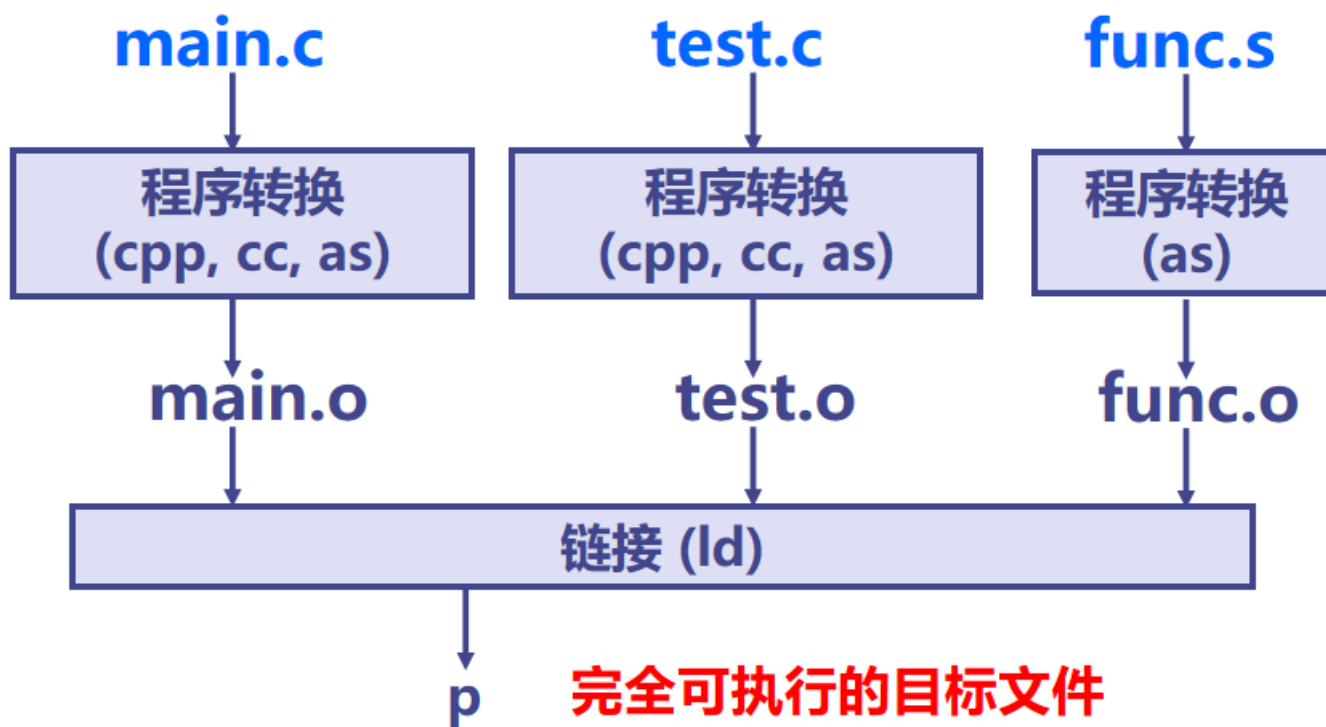
执行编译命令: gcc -c -
m32 -o test.o test.c

执行链接命令: gcc -m32
-o test main.o test.o



二、链接

将不同类型的源程序，编译、链接为一个执行程序：





二、链接

链接的任务：

1、符号解析

将符号的引用与一个确定的符号定义建立关联。

符号：全局变量名、函数名、静态的局部变量名

非静态的局部变量名不是符号。参数名不是符号。

- 编译器将所有符号存放在目标文件的符号表(symbol table)中。
- 符号表是一个结构数组，每个表项包含符号名、长度和位置信息等。



二、链接

链接的任务：

2、重定位

在合并生成执行文件时，重新确定每条指令的地址、每个数据的地址。

在指令中更新所引用符号对应的地址。



2、重定位

例：

观察前例中的main.o、test.o和test文件

objdump -d test.o

```
test.o:      file format elf32-i386
```

```
Disassembly of section .text:
```

```
00000000 <add>:
```

0:	55	push	%ebp
1:	89 e5	mov	%esp,%ebp
3:	83 ec 10	sub	\$0x10,%esp
6:	8b 55 08	mov	0x8(%ebp),%edx
9:	8b 45 0c	mov	0xc(%ebp),%eax
c:	01 d0	add	%edx,%eax
e:	89 45 fc	mov	%eax,-0x4(%ebp)
11:	8b 45 fc	mov	-0x4(%ebp),%eax
14:	c9	leave	
15:	c3	ret	



2、重定位

objdump -d main.o

```
main.o:      file format elf32-i386

Disassembly of section .text:

00000000 <main>:
  0:   8d 4c 24 04          lea    0x4(%esp),%ecx
  4:   83 e4 f0            and    $0xfffffffff0,%esp
  7:   ff 71 fc          push  -0x4(%ecx)
  a:   55                push  %ebp
  b:   89 e5            mov    %esp,%ebp
  d:   51                push  %ecx
  e:   83 ec 04          sub    $0x4,%esp
 11:   83 ec 08          sub    $0x8,%esp
 14:   6a 0d            push  $0xd
 16:   6a 14            push  $0x14
 18:   e8 fc ff ff ff    call  19 <main+0x19>
 1d:   83 c4 10          add    $0x10,%esp
 20:   8b 4d fc          mov    -0x4(%ebp),%ecx
 23:   c9                leave
 24:   8d 61 fc          lea    -0x4(%ecx),%esp
 27:   c3                ret
```



2、重定位

objdump -d test

```
0000117d <main>:
 117d:      8d 4c 24 04      lea    0x4(%esp),%ecx
 1181:      83 e4 f0        and    $0xffffffff0,%esp
 1184:      ff 71 fc        push   -0x4(%ecx)
 1187:      55              push   %ebp
 1188:      89 e5           mov    %esp,%ebp
 118a:      51              push   %ecx
 118b:      83 ec 04        sub    $0x4,%esp
 118e:      83 ec 08        sub    $0x8,%esp
 1191:      6a 0d           push   $0xd
 1193:      6a 14           push   $0x14
 1195:      e8 0b 00 00 00  call   11a5 <add>
 119a:      83 c4 10        add    $0x10,%esp
 119d:      8b 4d fc        mov    -0x4(%ebp),%ecx
 11a0:      c9              leave
 11a1:      8d 61 fc        lea    -0x4(%ecx),%esp
 11a4:      c3              ret
```

2、重定位



华中科技大学

objdump -d test

```
000011a5 <add>:
 11a5:    55                push    %ebp
 11a6:    89 e5             mov     %esp,%ebp
 11a8:    83 ec 10          sub     $0x10,%esp
 11ab:    8b 55 08          mov     0x8(%ebp),%edx
 11ae:    8b 45 0c          mov     0xc(%ebp),%eax
 11b1:    01 d0             add     %edx,%eax
 11b3:    89 45 fc          mov     %eax,-0x4(%ebp)
 11b6:    8b 45 fc          mov     -0x4(%ebp),%eax
 11b9:    c9               leave
 11ba:    c3               ret
```



主要内容



华中科技大学

- ◆ 4.1 编译、汇编和链接
- ◆ 4.2 目标文件格式
- ◆ 4.3 符号表和符号解析
- ◆ 4.4 重定位
- ◆ 4.5 可执行文件的加载





4.2. 目标文件格式

目标代码 (Object Code) : 机器语言代码

目标文件 (Object File) : 包含目标代码的文件

广义的目标文件:

- **可重定位目标文件: 编译或汇编输出的目标文件**
- **可执行目标文件: 链接输出的目标文件**

狭义的目标文件, 仅指可重定位目标文件。



4.2. 目标文件格式

一、目标文件的格式标准：

- **COM格式**：DOS操作系统提供的、最简单的目标文件格式。文件中仅包含代码和数据，且被加载到固定位置
- **COFF格式**：System V UNIX早期版本的目标文件格式。文件中不仅包含代码和数据，还包含重定位信息、调试信息、符号表等其他信息，由一组严格定义的数据结构序列组成（Common Object File Format）
- **PE格式**：Windows实现的COFF格式的变种，称为可移植可执行（Portable Executable），简称PE
- **ELF格式**：Linux等类UNIX实现的COFF格式的变种，称为可执行可链接（Executable and Linkable Format），简称ELF





二、目标文件的格式

可重定位目标文件和可执行目标文件，可以看作是目标文件的两种视图：

- **链接视图（被链接）**
- **执行视图（被执行）**

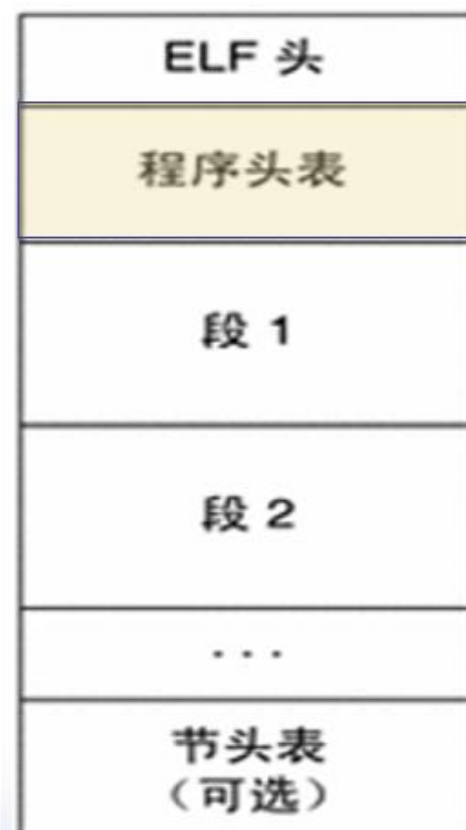


二、目标文件的格式

链接视图：可重定位目标文件



执行视图：可执行目标文件





二、目标文件的格式

可重定位目标文件主要由不同的节(section)组成。不同的节描述了目标文件中不同类型的信息及其特征。

例:

- .text: 代码节
- .rodata: 只读数据节
- .data: 已初始化全局数据节
- .bss: 未初始化全局数据节
- .symtab节:

符号表(symbol table)。符号包括函数名、全局变量名。符号表保存与这些符号相关的信息。每个可重定位目标文件都有一个.symtab节。





二、目标文件的格式

- **.rel.text节**

.text节相关的可重定位信息。

- **.rel.data节**

.data节相关的可重定位信息。

- **.debug节**

调试用符号表。带-g选项的gcc命令会得到这张表。

- **.line节**

C源程序中的行号和.text节中机器指令之间的映射。带-g选项的gcc命令会得到这张表。

- **.strtab节**

字符串表，包括.symtab节和.debug节中的**符号名**以及节头表中的节名字符串。





二、目标文件的格式

可执行目标文件由不同的段 (segment) 组成, 描述节如何映射到存储段中。可多个节映射到同一段。如: 可合并.data节和.bss节,并映射到一个可读可写数据段中。



二、目标文件的格式

程序头表用来指示系统如何创建进程的存储器映像。可执行文件和共享库文件必须具有程序头表，而可重定位目标文件无需程序头表。

节头表包含文件中各节的说明信息，每个节有一项，每一项指定了节名和节大小之类的信息。可重定位目标文件必须具有节头表。

主要内容



华中科技大学

- ◆ 4.1 编译、汇编和链接
- ◆ 4.2 目标文件格式
- ◆ 4.3 符号表和符号解析
- ◆ 4.4 重定位
- ◆ 4.5 可执行文件的加载





4.3.符号表和符号解析

一、符号和符号表

每个可重定位目标文件都有一个符号表，它包含了在该模块中定义的符号。有三种符号：

- **Global symbols: 全局符号**
- **External symbols: 外部符号**
- **Local symbols: 局部符号**



一、符号和符号表

- **Global symbols: 全局符号**

由该模块定义并能被其他模块引用的符号，包括函数和全局变量

。

- **External symbols: 外部符号**

由其他模块定义并被该模块引用的全局符号

注：在由多个模块组成的一个程序中，每个外部符号，都应该有一个对应的全局符号。否在链接时，会报“外部符号未定义”的错误

。

- **Local symbols: 局部符号**

由该模块定义，且仅由该模块引用的符号。例如在模块中定义的带static的函数和全局变量，带static的局部变量。

注：程序中的局部变量不是局部符号





一、符号和符号表

符号表 (symtab) 中每个条目的结构如下:

```
typedef struct {  
    int    name; /*符号对应字符串在strtab节中的偏移量*  
    int    value; /*在对应节中的偏移量, 可执行文件中是虚拟地  
址*/  
    int    size; /*符号对应目标所占字节数*/  
    char    type: 4, /*符号对应目标的类型: 数据、函数、源文件  
、节*/  
           binding: 4; /*符号类别: 全局符号、局部符号、弱符  
号*/  
    char    reserved;  
    char    section; /*符号对应目标所在的节, 或其他情况*/  
} Elf_Symbol;
```





一、符号和符号表

例:

```
# main.c
int buf[2] = {1, 2};
extern void swap();
int main()
{
    swap();
    return 0;
}
```

```
# swap.c
extern int buf[];
int *bufp0 = &buf[0];
static int *bufp1;
void swap()
{
    int temp;
    bufp1 = &buf[1];
    temp = *bufp0;
    *bufp0 = *bufp1;
    *bufp1 = temp;
}
```





一、符号和符号表

main.c 中的全局变量名buf为全局符号

main.c 中的函数名swap为外部符号

swap.c中的外部变量名buf为外部符号

swap.c中的函数名swap为全局符号

swap.c中的全局变量名bufp0为全局符号

swap.c 中的static变量名bufp1为局部符号

swap.c中，函数swap中的局部变量temp，不是局部符号。



一、符号和符号表

main.o中的符号表中最后三个条目 (共10个) (readelf -s main.o)

Num:	value	Size	Type	Bind	Ot	Ndx	Name
8:	0	8	Data	Global	0	3	buf
9:	0	33	Func	Global	0	1	main
10:	0	0	Notype	Global		0	UND

swap

buf是main.o中第3节 (.data) 偏移为0的符号, 是全局变量, 占8B;

main是第1节 (.text) 偏移为0的符号, 是全局函数, 占33B;

swap是main.o中未定义全局 (在其他模块定义) 符号, 类型和大小未知





一、符号和符号表

swap.o中的符号表中最后4个条目 (共11个) (readelf -s swap.o)

Num:	value	Size	Type	Bind	Ot	Ndx	Name
8:	0	4	Data	Global	0	3	bufp0
9:	0	0	Notype	Global	0	UND	buf
10:	0	36	Func	Global	0	1	swap
11:	4	4	Data	Local	0	COM	bufp1

bufp1是未分配地址且未初始化的本地变量(Ndx=COM), 按4B



对齐且占4B



二、符号解析

将每个模块中引用的符号与某个目标模块中的定义符号建立关联。

每个定义符号在代码段或数据段中都被分配了存储空间，将引用符号与定义符号建立关联后，就可在重定位时将引用符号的地址重定位为相关联的定义符号的地址。



二、符号解析

1.全局符号的强弱性

可能存在多重定义符号。即不同模块间，可能有多个同名的符号定义。

对于多重定义符号：

- 强符号：函数名，已初始化的全局变量名。
- 弱符号：未初始化的全局变量。



二、符号解析

多重定义符号的处理规则：

- 规则1：同名符号定义，若有多个强符号，则链接时报错。
- 规则2：同名符号定义，若有一个强符号和多个弱符号，则链接选强符号作为符号定义。
- 规则3：同名符号定义，若只有多个弱符号，则链接时任选其中一个。

二、符号解析

规则1示例:

```
int x=10;  
int p1(void);  
int main()  
{  
    x=p1();  
    return x;  
}
```

a) main.c文件

```
int x=20;  
int p1()  
{  
    return x;  
}
```

b) p1.c文件

模块main.c和p1.c中，符号x定义了两次，且都是强符号，所以链接时报错。



二、符号解析

规则2、3示例：

```
# include <stdio.h>
int y=100;
int z;
void p1(void);
int main()
{
    z=1000;
    p1();
    printf("y=%d, z=%d\n", y, z);
    return 0;
}
```

a) main.c文件

```
int y;
int z;

void p1()
{
    y=200;
    z=2000;
}
```

b) p1.c文件

模块main.c和p1.c中，符号y定义了两次：在main.c中是强符号，在p1.c中是弱符号。按规则2，链接器选择main.c中的符号y作为符号定义，p1.c中的符号y，作为符号引用。main函数输出为200。





二、符号解析

模块**main.c**和**p1.c**中，符号**z**也定义了两次，且两个定义都是弱符号。按规则**3**，链接器任意选择一个作为符号定义，其余的作为符号引用。例如可选择**main.c**中的符号**z**作为符号定义，**p1.c**中的符号**z**，作为符号引用。**main**函数输出为**2000**。

上述例子说明，如果在两个不同模块中，定义了同名的变量，很可能链接器不报错，但会产生意想不到的结果。





二、符号解析

2. 符号解析过程

首先定义三个集合：

- 集合E：可重定位目标文件集合。指的是被合并到一起，最终组成可执行文件的所有目标文件的集合。
- 集合U：未解析符号的集合。存储未解析符号，即还没有和对应符号关联起来的引用符号，即“只被引用，还没找到定义”的符号。
- 集合D：已定义符号的集合。存储当前已经加入到E中的所有目标文件里，定义过的符号。





二、符号解析

符号解析过程涉及的对象：

- 输入文件
- 符号本身



二、符号解析

- **输入文件：**

链接器按命令行顺序处理的输入，分为两类：

- **目标文件 (.o)：** 直接加入集合E，同时更新U、D
- **静态库文件 (.a)：** 用库中模块的定义符号，匹配U中的未解析符号，匹配成功，则将模块加入E、更新U/D。





二、符号解析

- **符号本身：**

符号本身也分为两类：

- **定义符号：变量/函数的定义，会被加入集合D；**
- **引用符号：对变量/函数的应用，未匹配定义时，会留在集合U。**



二、符号解析

具体解析过程如下：

1.初始状态：E、U、D 全部为空；

2.遍历输入文件：

若为目标文件：加入 E，用其定义符号更新 D，用其引用符号更新 U；

若为库文件：用库中模块的定义符号匹配 U 中的未解析符号，匹配成功则将模块加入 E、把符号从 U 移入 D，直到 U/D 不再变化；

3.最终校验：

若 D 中出现重复定义符号，或扫描完所有文件后 U 非空（有未解析符号），链接器报错；

校验通过则合并 E 中所有目标文件，生成可执行文件。





三、静态库的生成和使用

Linux中，静态库文件采用存档档案(archive)的文件格式，文件后缀为.a。





三、静态库的生成和使用

例：假定有两个源文件myproc1.c和myproc2.c,

```
// myproc1.c
```

```
#include <stdio.h>
```

```
void myfunc1()
```

```
{
```

```
    printf("%s","This is myfunc1 from mylib!\n");
```

```
}
```

```
// myproc2.c
```

```
#include <stdio.h>
```

```
void myfunc2()
```

```
{
```

```
    printf("%s","This is myfunc2 from mylib!\n");
```

```
}
```





三、静态库的生成和使用

生成静态库的过程：

```
gcc -c myprocl.c myproc2.c
```

```
ar rcs mylib.a myproc1.o myproc2.o
```



三、静态库的生成和使用

例：程序main.c使用了mylib.a中的函数myfunc1

```
void myfunc1(viod);
```

```
int main()
```

```
{
```

```
    myfunc1();
```

```
    return 0;
```

```
}
```



三、静态库的生成和使用

main.c的编译链接过程：

```
gcc -c main.c
```

```
gcc -static -o myproc main.o ./my1ib.a
```

注：命令中使用-static选项指示链接器应生成一个静态链接的可执行目标文件。

主要内容



华中科技大学

- ◆ 4.1 编译、汇编和链接
- ◆ 4.2 目标文件格式
- ◆ 4.3 符号表和符号解析
- ◆ 4.4 重定位
- ◆ 4.5 可执行文件的加载





4.4.重定位

链接器完成符号解析后，进入重定位过程。此过程分
两步：

- 重定位节和符号定义
- 重定位节中的符号引用



4.4.重定位

- **重定位节和符号定义**

将相同类型的节合并为新的聚合节，如将所有输入模块的.data节合并为输出可执行目标文件的.data节。链接器分配运行时存储器地址给新的聚合节、输入模块定义的每个节以及每个符号。此步完成后，每条指令和全局变量都有唯一的运行时存储器地址。

- **重定位节中的符号引用**

链接器修改代码和数据节中每个符号的引用，使其指向正确的运行时地址。





一、重定位信息

对于未知位置的目标引用，汇编器生成重定位表目 (relocation entry)，指示链接器如何修改符号引用。代码节的重定位表目在.relo.text中，数据节的重定位表目在.relo.data中。





一、重定位信息

ELF中重定位条目格式如下:

```
typedef struct {  
    int offset;    /*节内偏移*/  
    int symbol:24, /*所绑定符号*/  
    type: 8;       /*重定位类型*/  
} Elf32_Rel;
```

"offset"是需要修改的引用的节偏移

"symbol"标识引用应指向的符号

"type"指示链接器如何修改新引用

重定位类型主要有两种:

- R_386_PC32: 重定位使用32位PC相关地址引用
- R_386_32: 重定位使用32位绝对地址引用





一、重定位信息

例:

```
$ readelf -r main.o

Relocation section '.rela.text' at offset 0x208 contains 1 entries:
  Offset          Info           Type           Sym. Value      Sym. Name + Addend
00000000000005    000b00000002  R_X86_64_PC32  0000000000000000 _Z4funcv - 4

Relocation section '.rela.eh_frame' at offset 0x220 contains 1 entries:
  Offset          Info           Type           Sym. Value      Sym. Name + Addend
00000000000020    000200000002  R_X86_64_PC32  0000000000000000 .text + 0
```



一、重定位信息

```
$ readelf -r main.o

Relocation section '.rela.text' at offset 0x208 contains 1 entries:
  Offset          Info          Type           Sym. Value      Sym. Name + Addend
00000000000005  000b00000002  R_X86_64_PC32  0000000000000000  _Z4funcv - 4

Relocation section '.rela.eh_frame' at offset 0x220 contains 1 entries:
  Offset          Info          Type           Sym. Value      Sym. Name + Addend
00000000000020  000200000002  R_X86_64_PC32  0000000000000000  .text + 0
```

Offset表示需要修改的符号引用的位置。对于可重定位目标文件，该字段表示需要修改的符号引用的起始位置在目标 section 中的偏移量。

Info表示符号表索引和重定位类型，符号表索引占用高 32 位，重定位类型占用低 32 位。

符号表索引表示需要修改的符号引用在.symtab section中的索引。这里的 Sym. Value和Sym. Name列给出了所对应符号表条目中Value和Name列的值。

重定位类型指示链接器如何修改该符号引用的值。重定位类型因不同的处理器而异。

Addend表示一个有符号常数，一些重定位类型要使用它对被修改符号引用的值做偏移调整。





二、重定位过程

重定位过程分为两步：

- 阶段1：合并节（空间分配）
- 阶段 2：地址重定位（修正引用）



合并节（空间分配）

链接器将所有输入目标模块（main.o、swap.o）的同类节合并，为每个节分配运行时虚拟地址：

1.合并.text节：将main.o的main()和swap.o的swap()代码合并，分配连续的代码段地址（如main从0x8048380开始，swap从0x8048394开始）

2.合并.data节：将main.o的buf数组和swap.o的bufp0变量合并，分配数据段地址（如buf从0x8049620开始，bufp0从0x8049628开始）

3.合并.bss节：为未初始化的静态变量（如bufp1）分配地址（如0x8049700）

4.为每个符号分配最终运行时地址：

ADDR(swap)=0x8048394、ADDR(buf)=0x8049620、
ADDR(bufp0)=0x8049628等





地址重定位（修正引用）

遍历每个目标模块的重定位表（.rel.text/.rel.data），
根据重定位类型修正代码 / 数据中的地址引用。

重定位类型分为两种：

- R_386_PC32（PC 相对地址重定位，用于函数调用）
- R_386_32（绝对地址重定位，用于变量访问）

R_386_PC32 (PC 相对地址重定位)



华中科技大学

代码段 (.text) 中的函数调用指令 (如call swap) , 这类指令使用PC 相对寻址 (目标地址 = 当前 PC + 偏移量) 。

- 重定位值 = $\text{ADDR}(r_sym) - (\text{ADDR}(.text) + r_offset) - \text{init})$
- $\text{ADDR}(r_sym)$: 引用符号的最终地址 (如swap的0x8048394)
- $\text{ADDR}(.text) + r_offset$: 需要重定位的call指令的地址 (如 $0x8048380 + 0x7 = 0x8048387$)

init: 重定位位置的初始值



R_386_32 (绝对地址重定位)



华中科技大学

用于:

- 数据段 (.data) 中的变量地址初始化 (如 `bufp0 = &buf`)
- 代码段 (.text) 中对全局变量的直接访问 (如 `*bufp0`、
`bufp1 = &buf[1]`)

这类引用需要直接写入符号的绝对运行时地址。

- 重定位值 = `init + ADDR(r_sym)`

`init`: 重定位位置的初始值 (通常为0, 如 `bufp0` 初始为 `0x0`)

`ADDR(r_sym)`: 引用符号的最终地址 (如 `buf` 的 `0x8049620`)





4.5.可执行文件的加载

通过调用execve系统调用函数来调用加载器

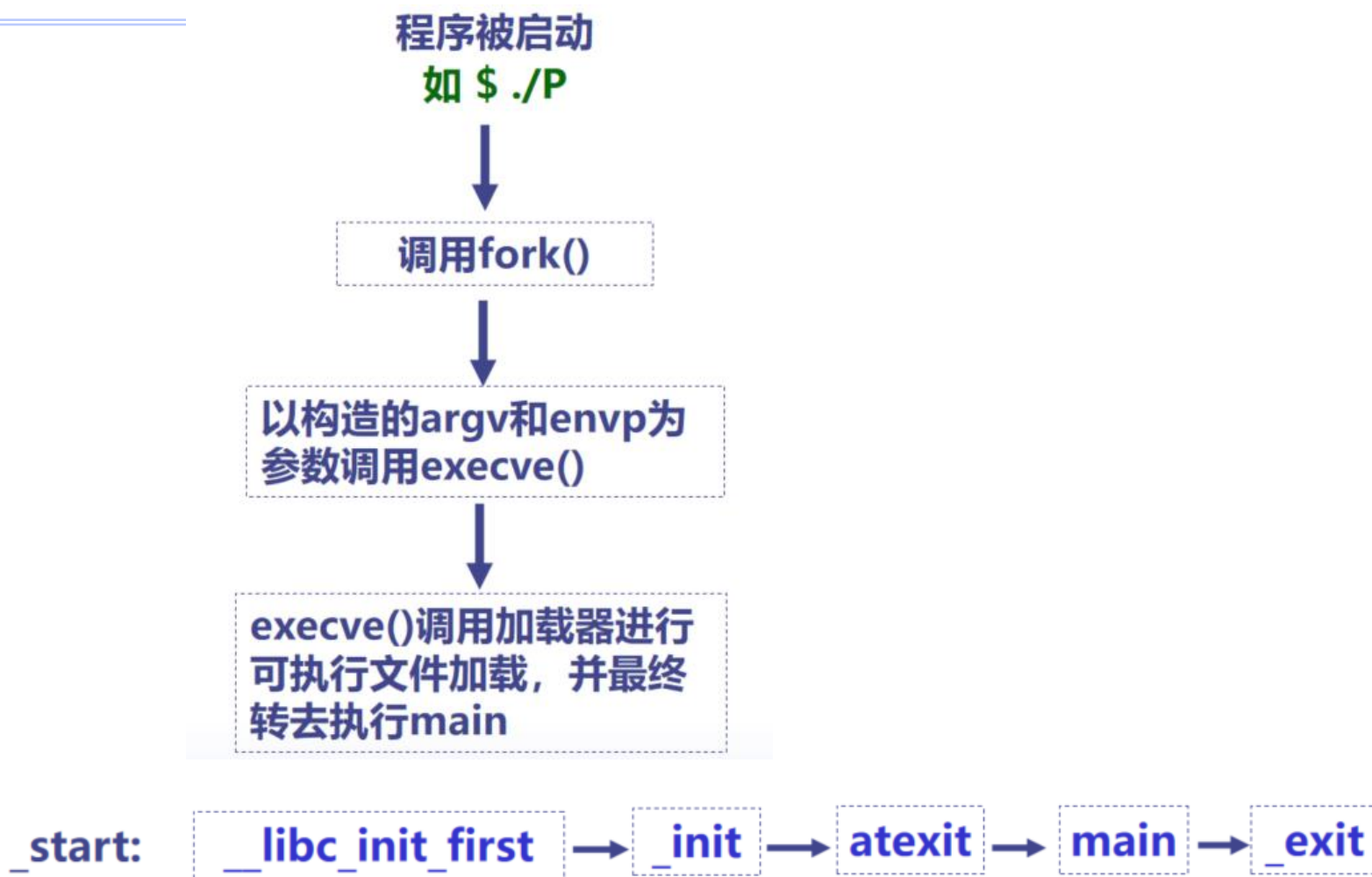
加载器 (loader) 根据可执行文件的程序 (段) 头表中的信息, 将可执行文件的代码和数据从磁盘“拷贝”到存储器中 (实际上不会真正拷贝, 仅建立一种映像, 这涉及到许多复杂的过程和一些重要概念, 将在后续课上学习)

加载后, 将PC (EIP) 设定指向Entry point (即符号_start处), 最终执行main函数, 以启动程序执行。





4.5.可执行文件的加载





4.5.可执行文件的加载

可执行文件的存储器映像：

